Complete Hardware Assignment (SystemVerilog)

Sreejit Maity

October 2026

Introduction

We want to design a 4-bit Arithmetic Logic Unit (ALU) using SystemVerilog with an internal register, and verify its functionality using a simple C cycle simulation.

Design Specifications

- **Inputs:** 4-bit A, 4-bit B, 2-bit operation selector (`alu_op`), `clk`, and `rst`.
- **Outputs:** 4-bit result, Carry flag, Zero flag.
- **Operations:**
  - `2'b00`: Addition ($A + B$)
  - `2'b01`: Subtraction ($A - B$)
  - `2'b10`: Bitwise AND (A & B)
  - `2'b11`: Bitwise XOR (A ^ B)

SystemVerilog Module

```
module alu_4bit (
    input  logic       clk,
    input  logic       rst,
    input  logic [3:0] a,
    input  logic [3:0] b,
    input  logic [1:0] op,
    output logic [3:0] result,
    output logic       zero_flag,
    output logic       carry_flag
);

    logic [4:0] temp_calc;

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            result     <= 4'b0000;
            carry_flag <= 1'b0;
            zero_flag  <= 1'b1;
        end else begin
            case (op)
                2'b00: temp_calc = a + b;           // ADD
                2'b01: temp_calc = a - b;           // SUB
                2'b10: temp_calc = {1'b0, (a & b)}; // AND
                2'b11: temp_calc = {1'b0, (a ^ b)}; // XOR
```

```
                default: temp_calc = 5'b00000;
            endcase

            result     <= temp_calc[3:0];
            carry_flag <= temp_calc[4];
            zero_flag  <= (temp_calc[3:0] == 4'b0000);
        end
    end
endmodule
```

C Cycle Simulator

This C code mirrors the clock-by-clock behavior of our hardware design.

```
#include <stdio.h>

typedef struct {
    unsigned char result;
    unsigned char carry_flag;
    unsigned char zero_flag;
} ALU_Output;

ALU_Output run_alu_cycle(unsigned char a, unsigned char b, unsigned char op) {
    ALU_Output out;
    unsigned short temp = 0;

    a &= 0x0F; // Keep 4 bits
    b &= 0x0F;

    if (op == 0)      temp = a + b;
    else if (op == 1) temp = (a - b) & 0x1F;
    else if (op == 2) temp = (a & b);
    else if (op == 3) temp = (a ^ b);

    out.result = temp & 0x0F;
    out.carry_flag = (temp >> 4) & 1;
    out.zero_flag = (out.result == 0);

    return out;
}

int main() {
    printf("--- Running 4-bit ALU Testbench in C ---\n");

    unsigned char a = 9, b = 7;
    // Operation 0: Addition (9 + 7 = 16 -> Overflow/Carry out)
    ALU_Output r1 = run_alu_cycle(a, b, 0);
    printf("ADD (9+7): Result = %d, Carry = %d, Zero = %d\n",
           r1.result, r1.carry_flag, r1.zero_flag);

    // Operation 3: XOR (9 ^ 9 = 0)
    ALU_Output r2 = run_alu_cycle(a, a, 3);
    printf("XOR (9^9): Result = %d, Carry = %d, Zero = %d\n",
           r2.result, r2.carry_flag, r2.zero_flag);

    return 0;
}
```

Output

— Running 4-bit ALU Testbench in C —
ADD ($9 + 7$): Result = 0, Carry = 1, Zero = 1
XOR (9^9): Result = 0, Carry = 0, Zero = 1

Conclusion

The design successfully models basic synchronous arithmetic behavior. SystemVerilog synthesizes this directly into flip-flops and lookup tables on an FPGA.